

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

SUSTAV ZA PRAĆENJE I ODRŽAVANJE KAKVOĆE VODE U AKVARIJU

Matko Marjanac

Zagreb, srpanj 2024.

Student:

Matko Marjanac

Naslov (na hrvatskom):

Sustav za praćenje i održavanje kakvoće vode u akvariju

Naslov (na engleskom):

System for monitoring and water quality maintenance in the aquarium

Studij:

Elektrotehnika i informacijska tehnologija

Mentor:

prof. dr. sc. Hrvoje Džapo

Kratki opis:

Upoznati se s arhitekturom i mogućnostima mikrokontrolera iz porodice ESP32. Proučiti mogućnosti i način korištenja programskog razvojnog okvira ESP-IDF. Proučiti i odabrati senzore prikladne za mjerenje temperature vode i određivanje prisutnosti otopljenih tvari (Total Dissolved Solids, TDS). Realizirati sklopovski prototip sustava za praćenje i održavanje kakvoće vode u akvariju temeljen na sensorima (temperatura, TDS) i aktuatorima (npr. pumpa za cirkulaciju vode kroz filter). Razviti programsku potporu za mikrokontroler iz porodice ESP32 koja uključuje: prikupljanje podataka sa senzora, upravljanje aktuatorima i povezivanje sustava putem bežičnih komunikacijskih sučelja (WiFi i BLE). Izraditi cjelokupni postav sustava te razviti korisničko sučelje za prikaz izmjerenih parametara vode u akvariju i upravljanje aktuatorima.

Zahvaljujem profesoru Džapi i asistentici Salamun na pomoći pri izradi ovog rada

Sadržaj

1.	Uvod	1
2.	Upravljanje kakvoćom vode u akvariju.....	2
2.1.	Temperatura	2
2.2.	Zamućenost vode.....	3
2.3.	Ciljevi rada.....	3
3.	Sklopovlje	4
3.1.	ESP32 mikrokontroler	4
3.1.1.	Razvojna pločica.....	5
3.2.	DS18B20 temperaturni senzor	7
3.3.	SEN0189 senzor zamućenosti vode	8
4.	Programska podrška	10
4.1.	1-Wire protokol i komunikacija sa senzorom DS18B20	10
4.2.	SEN0189 senzor kod	16
4.2.1.	A/D pretvornik (ADC).....	16
4.2.2.	Implementacija filtra za senzor zamućenosti	17
4.3.	Upravljanje aktuatorima	19
4.3.1.	LEDC	20
5.	Bežično slanje mjerenih podataka.....	22
5.1.	Wi-Fi.....	22
5.2.	MQTT	24
5.2.1.	ESP-MQTT	25
5.3.	Azure.....	27
6.	Pokazni model.....	29
6.1.	Sklopovlje	29

6.2. Rezultati	30
7. Zaključak	31
Literatura	32
Sažetak	34
Summary	35

1. Uvod

Trend porasta broja vlasnika kućnih ljubimaca doveo je do povećanja potražnje za inovativnim rješenjima o njihovoj brizi. Jedan od popularnijih izbora za ljubitelje kućnih ljubimaca su akvariji [25]. Delikatna ravnoteža ekosustava tog umjetnog staništa zahtijeva stalnu pažnju njegova vlasnika. Korištenjem naprednih tehnologija olakšava se briga o kućnim ljubimcima. U tu svrhu nastaju pametni sustavi koji implementiraju bežične veze za njihovo udaljeno praćenje.

U radu će se izraditi jedan takav sustav koji integrira senzorsku tehnologiju s mehanizmima kontrole za ostvarivanje poboljšane kvalitete staništa i očuvanja zdravlja vodenih organizama. Razvijeni sustav kontinuiranim mjerenjem temperature i zamućenosti vode pružat će uvid u trenutno stanje akvarija. Obrada i bežično slanje prikupljenih podataka bit će zadaća implementiranog mikrokontrolera. Aktuatorski elementi osiguravat će pravovremenu regulaciju parametara vode, čime će se održavati optimalni uvjeti za život.

U prvom poglavlju govorit će se o najvažnijim parametrima akvarija kao staništa za vodeni život. Također, prikazat će se moderna rješenja za njegovo automatsko upravljanje. U drugom poglavlju detaljno su opisani korišteni mikrokontroler i senzori. Razvijena programska podrška, aktuatori i njihovo upravljanje obrađeni su u trećem poglavlju. Bežični prijenos mjerenih vrijednosti s uređaja na oblak (*engl. cloud*) i korišteni protokoli pri komunikaciji objašnjeni su u četvrtom poglavlju. Peto poglavlje pobliže opisuje realizirani pokazni model i govori o rezultatima samog projekta.

Razvoj sustava za praćenje i kontrolu parametara vode u akvarijima predstavlja značajan korak naprijed u tehnologiji brige o kućnim ljubimcima. Kako vlasništvo nad kućnim ljubimcima nastavlja rasti, potreba za pouzdanim i korisnički prilagođenim rješenjima postaje sve očitija.

2. Upravljanje kakvoćom vode u akvariju

Kvaliteta vode u akvariju ovisi o nekoliko ključnih vrijednosti koje je važno redovito pratiti. Prvo, pH vrijednost vode treba biti odgovarajuća za vrste riba u održavanom staništu, jer previše kisela ili alkalna voda može biti štetna. Također, razina amonijaka, nitrita i nitrata mora biti niska, jer visoke koncentracije ovih tvari mogu biti toksične za ribe. Konačno, tvrdoća vode i koncentracija otopljenog kisika su također važni parametri za održavanje zdravog akvarijskog ekosustava. Posebno se izdvajaju temperatura i mjera zamućenosti vode kao dva parametra korištena u projektu. Moderni sustav za upravljanje kakvoćom vode, kakav je prikazan slikom (Slika 1), uspostavlja kontrolu nad svim navedenim parametrima i osigurava optimalno stanište za vodeni život.



Slika 1. Moderni sustav upravljanja kakvoćom vode tvrtke Intaqo. [5][6]

2.1. Temperatura

Osnovna zadaća sustava za upravljanje kakvoćom vode u akvariju je kontinuirano mjeriti i održavati parametar s najvećim utjecajem na vodeni život, temperaturu vode. Napuštanje optimalnog temperaturnog raspona u riba dovodi do promjena u metabolizmu, motoričkim sposobnostima i obrascima prehrane te često nosi kobne posljedice [15].

Posljedica razlike u temperaturi vode između pojedinih dijelova akvarija jest potreba za spajanjem većeg broja temperaturnih senzora. U realiziranom sustavu koristi se poluvodički temperaturni senzor s mogućnosti realizacije proizvoljno dugog mjernog lanca.

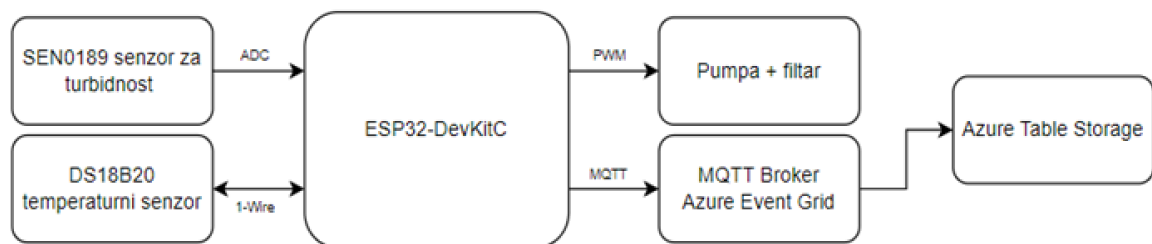
2.2. Zamućenost vode

Visoka zamućenost vode može imati brojne negativne učinke na zdravlje i dobrobit riba. Zamućenost vode uzrokuje smanjenu vidljivost, što može uzrokovati stres kod riba jer im otežava pronalaženje hrane i izbjegavanje opasnosti. Dodatno, zamućena voda često znači prisutnost čestica koje mogu iritirati škrge riba, otežavajući im disanje. Osim toga, zamućenost može biti pokazatelj loše kvalitete vode i prisutnosti štetnih tvari, poput otpada ili prekomjerne količine algi [16].

Kontinuirano praćenje i regulacija spomenutog parametra su od velike važnosti za održavanje optimalnih uvjeta staništa.

2.3. Ciljevi rada

Cilj rada je ostvariti robustan sustav za praćenje i održavanje kakvoće vode u pokaznom modelu. Sustav će implementirati temperaturni senzor DS18B20 i senzor za mjerenje zamućenosti vode SEN0189. Koristit će se mikrokontroler ESP32 pomoću kojeg će se čitati i obrađivati mjerene vrijednosti, upravljati aktuatorskim elementima i bežično slati podatci. Poslani podatci skladištiti će se u oblaku koristeći usluge Azure platforme. Blok shema ciljnog sustava dana je slikom (Slika 2).

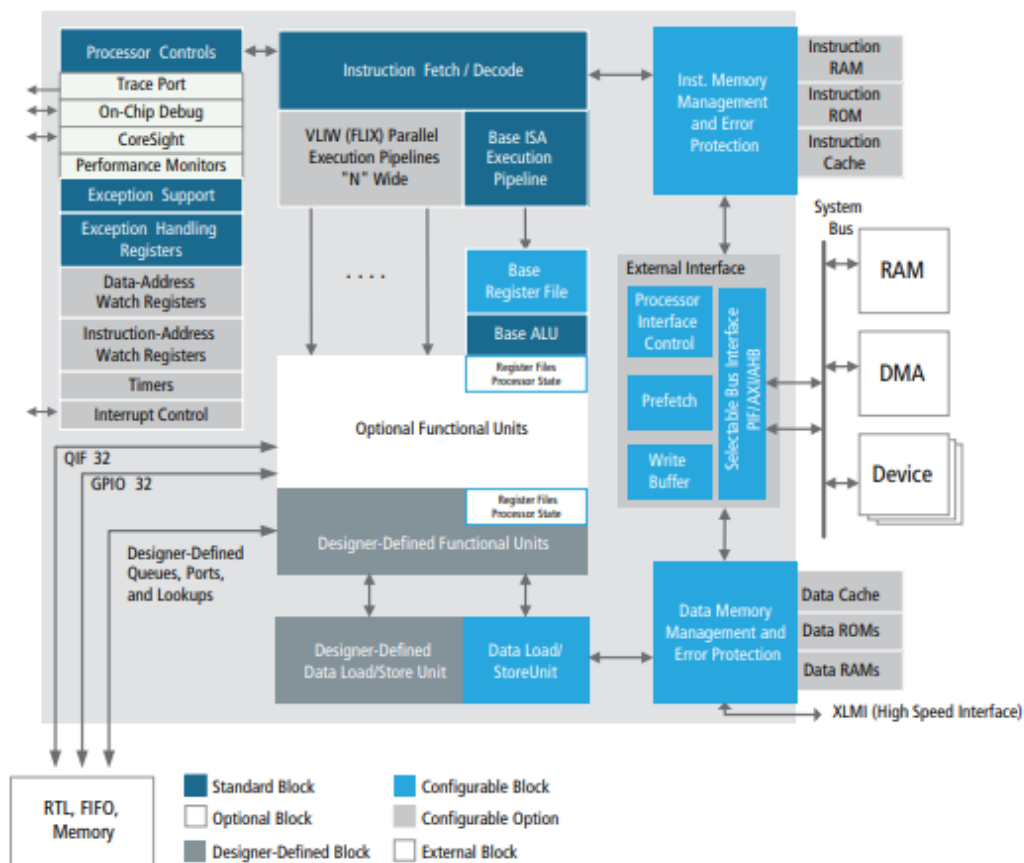


Slika 2. Blok shema realiziranog sustava

3. Sklopovlje

3.1. ESP32 mikrokontroler

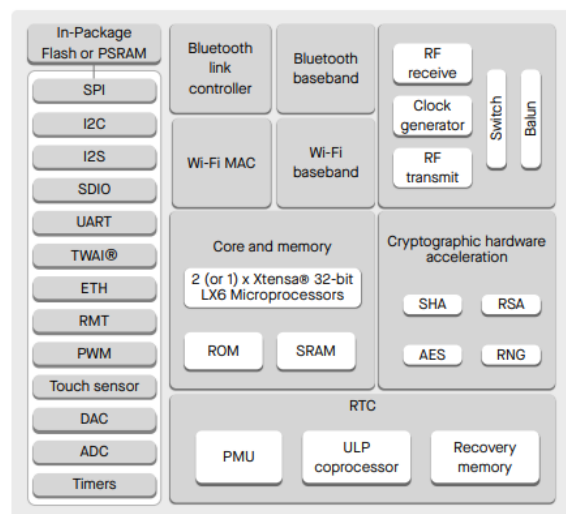
ESP32 je porodica mikrokontrolera tvrtke Espressif. Sadrži integrirano Wi-Fi sučelje, *dual-mode* Bluetooth i popratno RF sklopovlje. Mikrokontroler dolazi u brojnim varijacijama s jednom i dvije Xtensa LX6 procesorske jezgre tvrtke Tensilica. Xtensa LX6 je mikroprocesorska jezgra temeljena na Xtensa procesorskoj arhitekturi. 32-bitna RISC arhitektura, 16- i 24-bitne instrukcije, promjenjivi broj razina protočne strukture (5- ili 7-razinska), 64 registra opće i 6 registara specifične namjene su neke od karakteristika spomenute jezgre. Blok shema procesorske jezgre prikazana je slikom 3 (Slika 3).



Slika 3. Blok shema Xtensa LX6 procesorske jezgre. Preuzeto iz [4].

Neke karakteristike mikrokontrolera porodice ESP32 su:

- 32-bitni procesor, 160/240 MHz
- ULP (*ultra low power*) koprocesor namijenjen za način rada niske potrošnje
- 448 KB ROM, 520 KB SRAM, 16 KB SRAM u RTC sklopu
- interni 8MHz oscilator, interni RC (150 kHz) oscilator, eksterni 2MHz – 60MHz kristalni oscilator, vanjski 32 kHz oscilator za RTC.
- Wi-Fi s mogućnosti korištenja 802.11b/g/n inačica norme IEEE 802.11. Maksimalna brzina prijenosa ostvarena je prilikom korištenja 802.11n norme i iznosi 150 Mbit/s.
- Bluetooth v4.2 BR/EDR i BLE (*Bluetooth low energy*).
- periferije: 12-bit SAR ADC, 2x 8-bit DAC, 4x SPI, 2x I²S, 2x I²C, 3x UART, PWM generatori (LED i motor kontrola), 34 programabilna GPIO izvoda itd.



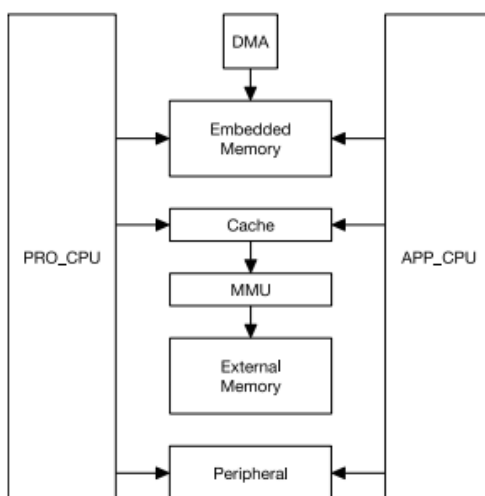
Slika 4. ESP32 blok shema. Preuzeto iz [22]

Mikrokontroleri porodice ESP32 pronalaze primjenu u nosivim elektroničkim uređajima, senzorskim čvorovima i drugim situacijama gdje je njihova mogućnost raznih načina rada, niske potrošnje i bežične komunikacije od velike važnosti.

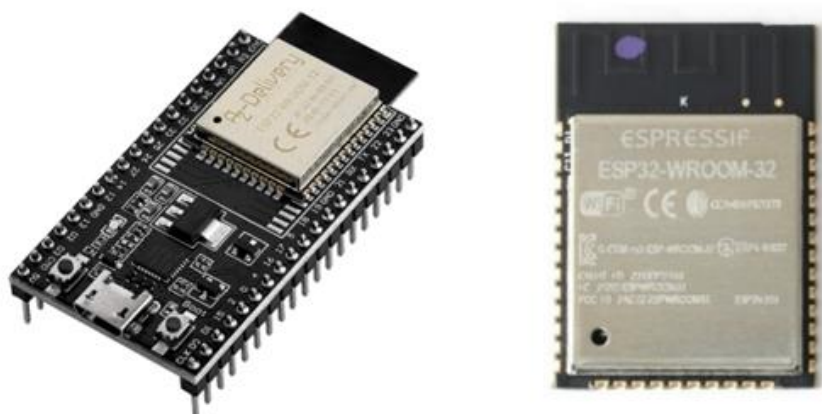
3.1.1. Razvojna pločica

Korištena ESP32-DevKitC razvojna pločica sadrži modul ESP32-WROOM-32E (Slika 6). Modul implementira mikrokontroler D0WDQ6 iz ESP32 porodice. Mikrokontroler sadrži dvije procesorske jezgre Xtensa LX6 („PRO_CPU“, „APP_CPU“) s implementiranom Harvard arhitekturom i simetričnim memorijskim prostorom (Slika 5). Vanjska SPI flash

memorija veličine 4MB spojena je na GPIO6, GPIO7, GPIO8, GPIO9, GPIO10 i GPIO11 izvode mikrokontrolera. Poželjno je izbjegavati korištenje navedenih izvoda zbog mogućnosti pojave smetnji prilikom čitanja naredbi ili podataka skladištenih u vanjskoj flash memoriji. Kristalni oscilator frekvencije 40 MHz spaja se između XTAL_N (vanjski kristal, *negative*) i XTAL_P(vanjski kristal, *positive*) izvoda mikrokontrolera. Signal se s ulaznih stezaljki povlači do sustava za razvođenje i modifikaciju frekvencije signala vođenja. Ugrađena PCB ili IPEX antena modula omogućuje odašiljanje i primanje elektromagnetskih valova u RF području. Dimenzije modula su: 18 mm x 25.5 mm x 3.10 mm.



Slika 5. Arhitektura D0WDQ6 mikrokontrolera. Preuzeto iz [22].



Slika 6. Razvojna pločica ESP32-DevKitC i modul ESP32-WROOM-32E. Preuzeto iz [3] [7].

3.2. DS18B20 temperaturni senzor

DS18B20 je digitalni senzor koji se sastoji od poluvodičkog temperaturnog senzora, analogno-digitalnog pretvornika, registara za konfiguraciju, privremene memorije, ROM i sklopovlja koje osigurava komunikaciju putem *1-Wire* protokola. Prikladan je za mjerenja na lokacijama udaljenim od mikrokontrolera i napajanja. Implementacija 1-Wire sučelja omogućuje formiranje mjernog lanca proizvoljnog broja senzora koji komuniciraju s mikrokontrolerom putem zajedničke linije. Vodonepropusno metalno kućište osigurava dobru toplinsku vodljivost i štiti senzor od vanjskih uvjeta.

Tablica 1. Specifikacija senzora DS18B20

Parametar	Simbol	Uvjet	Vrijednost	Mjerna jedinica
Greška	T_{ERR}	-10°C - 85°C	$\pm 0,5$	°C
<i>Drift</i>		+125°C, $V_{dd} = 5.5V$	$\pm 0,2$	°C
Struja u aktivnom stanju	I_{dd}	$V_{dd} = 5V$	1 – 1,5	mA
Struja u neaktivnom stanju	I_{dds}	$V_{dd} = 5V$, $T < 70\text{ °C}$	750 - 100	nA
Vrijeme pretvorbe	t_{CONV}	9-bit rezolucija	93,5	ms
		10-bit rezolucija	187,5	
		11-bit rezolucija	375	
		12-bit rezolucija	750	



Slika 7. DS18B20 u TO-92 paketu zaštićen vodonepropusnim kućištem

3.3. SEN0189 senzor zamućenosti vode

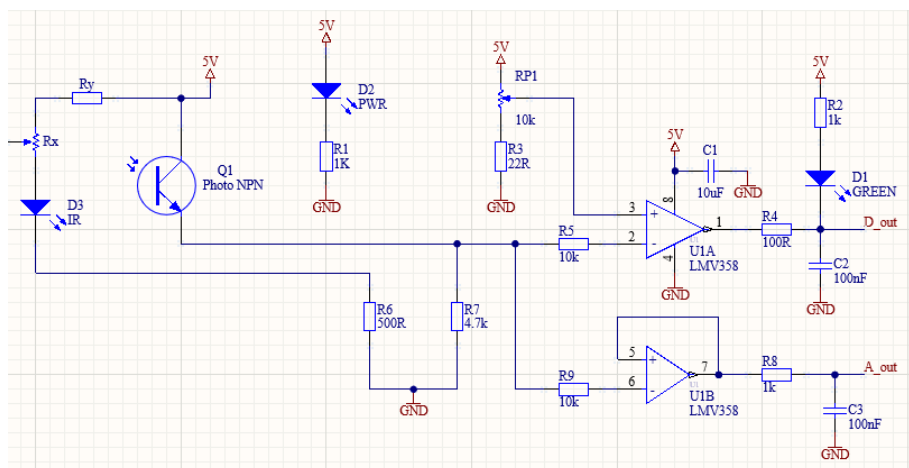
Turbidity ili mjera zamućenosti fluida je teško mjerljiv, ali važan parametar pri praćenju trenutnog stanja tijela vode. Pruža uvid u količinu suspendiranih čestica (*engl. Total Suspended Solids*, TSS) i prisutnost otopljenih tvari (*engl. Total Dissolved Solids*, TDS), dvije veličine s velikim utjecajem na održavano vodeno stanište.

Teško definiranje izmjerene vrijednosti zamućenosti vode za posljedicu ima postojanje nekoliko mjernih jedinica usko vezanih uz korištenu mjernu metodu. Nefelometrijske i formazinske jedinice za mutnoću (*engl. Nephelometric Turbidity Unit*, NTU i *engl. Formazine Turbidity Unit*, FTU) su dvije najčešće korištene mjerne jedinice za iskazivanje zamućenosti u industriji. Od brojnih mjernih metoda razvijenih kroz povijest izdvajaju se dvije u uporabi još i danas, Secchijev disk i mjerenje refleksije i transmisije infracrvene svjetlosti.

Korišteni senzor SEN0189 proizvođača DF robot temelji se na metodi mjerenja atenuacije svjetlosti bliske infracrvenoj u tekućini. Sastoji se od fotodiode, fototranzistora i popratnog sklopovlja za promjenu osjetljivosti, filtraciju i/ili digitalizaciju vrijednosti.

Senzor, kao i mnogi drugi namijenjeni za mjerenje zamućenosti, zahtjeva kontinuirano održavanje i umjeravanje. Umjeravanje se provodi po standardiziranom postupku koristeći tekućinu s poznatim vrijednostima zamućenosti (NTU/FTU), mješavinu demineralizirane vode i formazina.

Formazin je polimer slabo topljiv u vodi. Sastoji se od brojnih koloidalnih čestica koje su ravnomjerno raspoređene u otopini. Homogenost tekućine i ujednačene veličine čestica omogućuju stalnost mjerenja razine zamućenosti što je dovelo do njegove primjene diljem industrije. Različite vrijednosti standarda mogu se realizirati miješanjem osnovne otopine s demineraliziranom vodom u omjeru ovisnom o željenom NTU produkta.



Slika 8. Električna shema senzora SEN0189 senzora



Slika 9. NTU standardi. Preuzeto iz [1][2].



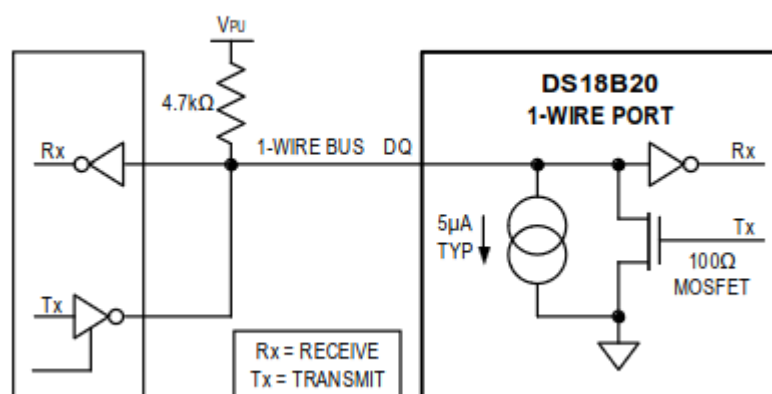
Slika 10. Senzor SEN0189

4. Programska podrška

4.1. 1-Wire protokol i komunikacija sa senzorom DS18B20

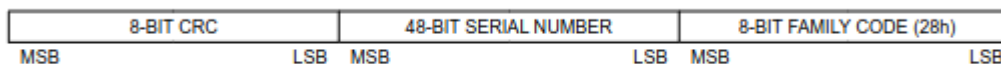
1-Wire je žični, polu-dvosmjerni serijski protokol sličan protokolu I2C namijenjen za niske brzine prijenosa i velike udaljenosti. Uređaji se na zajedničku sabirnicu, pritegnutu na visoku naponsku razinu preko priteznog otpornika, spajaju u *open-drain* konfiguraciji. Dodatno, potrebne su linije za napon napajanja i uzemljenje ili, ako se koristi parazitni način rada, samo linija za uzemljenje. Spajanje temperaturnog senzora DS18B20 s mikrokontrolerom preko zajedničke sabirnice prikazano je slikom 11 (Slika 11).

Parazitni način rada omogućuje mjerenja na lokacijama udaljenim od mikrokontrolera i vanjskih izvora napajanja. Tijekom neaktivnog stanja zajedničke sabirnice, struja kroz pritezni otpornik služi za punjenje kondenzatora na sklopu za parazitno napajanje i opskrbljuje uređaj. Tijekom prijenosa podataka sabirnicom, energija za normalan rad uređaja se crpi iz kondenzatora. Primjena parazitnog načina napajanja stavlja gornju granicu na broj povezanih uređaja i unosi dodatne komplikacije u izvođenju određenih dugotrajnijih operacija za koje energija skladištena u kondenzatoru nije dostatna.



Slika 11. DS18B20 spajanje na zajedničku sabirnicu. Preuzeto iz [19].

Svaki uređaj posjeduje jedinstveni, nepromjenjivi, tvornički programirani 48-bitni serijski kod koji se koristi za njegovo adresiranje od strane mikrokontrolera. Slika 12 prikazuje strukturu ROM-a unutar kojeg se nalazi serijski kôd.



Slika 12. DS18B20 ROM. Preuzeto iz [19].

Uređaji koji koriste 1-Wire sabirnicu moraju implementirati osnovni skup naredbi definiran protokolom.

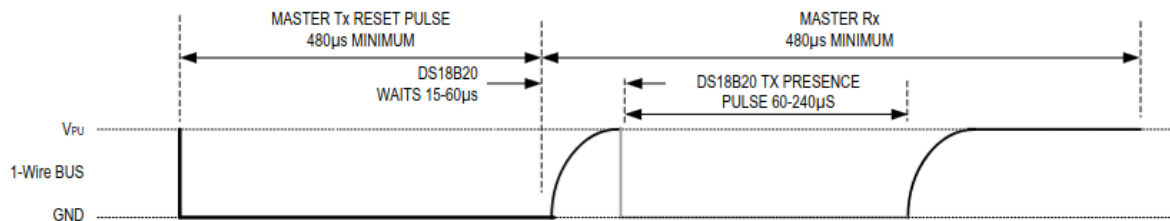
Tablica 2 sadrži imena, kôdove i funkcije naredbi. DS18B20 temperaturni senzor dodatne naredbe kojima se upravlja konfiguracijskim registrima, iniciraju temperaturna mjerenja i provode ostale operacije za upravljanje senzorom. Tablica 3 prikazuje sve naredbe senzora. Inicijalizaciju senzora provodi funkcija `init_DS()` definirana u `ds18b20.c` datoteci. Funkcija uspostavlja komunikaciju sa senzorom i iščitava njegov ROM kôd preko GPIO izvoda predanog kao parametar funkcije. Dodatno, modificira elemente pomoćne strukture `DS_Info` čiju adresu u memoriji prima putem parametra.

Tablica 2. Lista ROM naredbi

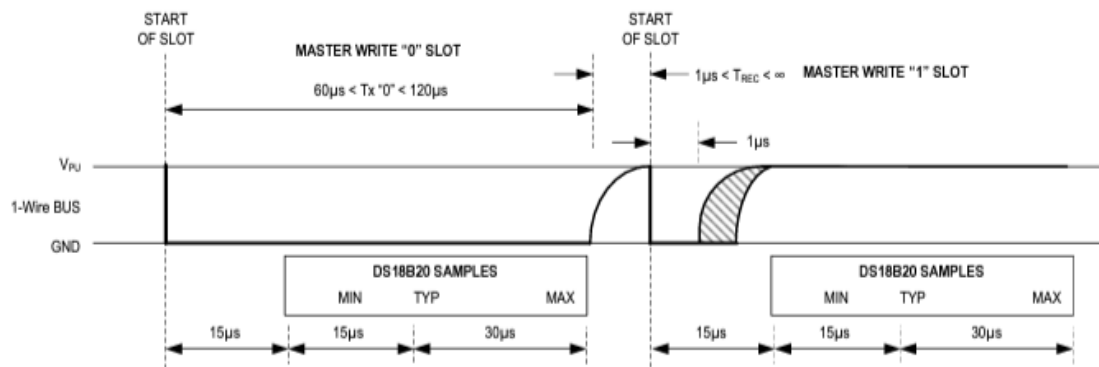
Ime naredbe	Kod naredbe	Opis
READ ROM	0x33	Pročitaj ROM uređaja
SKIP ROM	0xCC	Preskoči provjeru ROM-a
SEARCH ROM	0xF0	Pretraži sabirnicu i identificiraj nove uređaje
MATCH ROM	0x55	Adresiraj određeni uređaj
ALARM SEARCH	0xEC	Provjera zastavica uređaja

Tablica 3. Lista naredbi DS18B20 senzora

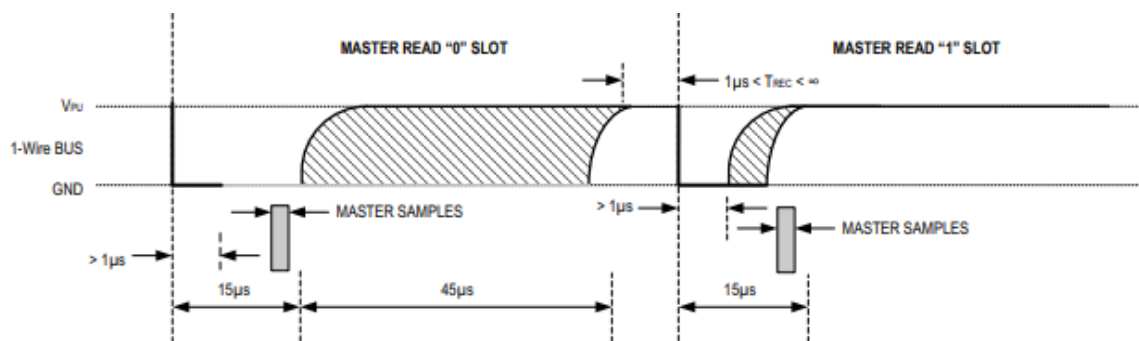
Ime naredbe	Kod naredbe	Opis
Convert T	0x44	Započni temperaturno mjerenje
Write scratchpad	0x4E	Pisanje u interni SRAM
Read scratchpad	0xBE	Čitanje internog SRAM-a
Copy scratchpad	0x48	Spremanje konfiguracije iz SRAM u EEPROM
Recall E ²	0xB8	Povlačenje konfiguracijskih registara iz EEPROM-a u SRAM
Read PS	0xB4	Način napajanja uređaja



Slika 13. Vremenski dijagram „Reset/Presence“ sekvence. Preuzeto iz [19].



Slika 14. Vremenski dijagram upisa bita na zajedničku sabirnicu. Preuzeto iz [19].



Slika 15. Vremenski dijagram čitanja bita sa zajedničke sabirnice. Preuzeto iz [19]

Komunikacija putem sabirnice uvijek slijedi tri osnovna koraka:

1. Inicijalizacijska ili „Reset/Presence“ sekvenca
2. Slanje ROM naredbe od strane mikrokontrolera i opcionalna razmjena podataka
3. Slanje funkcionalne naredbe uređaja od strane mikrokontrolera

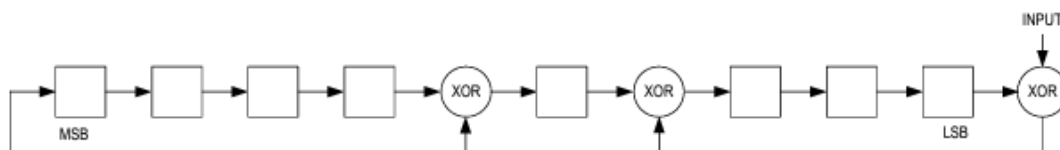
Komunikaciju započinje *master* uređaj slanjem signala za resetiranje. Nakon poslanog signala *master* uređaj počinje osluškivati liniju iščekujući odgovor uređaja spojenih na nju. Vremenski dijagram „Reset/Presence“ sekvence između *master* i *slave* uređaja prikazan je slikom 13 (Slika 13). Inicijalizacijska sekvenca implementirana je funkcijom `reset_sequence()`. Potvrdom prisutnosti uređaja na liniji mikrokontroler na zajedničku sabirnicu upisuje kôd željene ROM naredbe. Bajt podatka upisuje se na liniju pomoću funkcije `write_byte()` neposredno ovisne o funkciji `write_bit()`.

Implementacija funkcije za emitiranje bitova na liniju od strane mikrokontrolera mora zadovoljavati vremenske zahtjeve *1-Wire* protokola za dotičnu operaciju. Slika 14 prikazuje vremenski dijagram upisa jednog bita na sabirnicu. Mikrokontroler, nakon slanja jedne od naredbi definiranih *1-Wire* protokolom, čita što uređaj emitira na zajedničku liniju ili sam emitira instrukcije uređaju. Bajt podatka čita se sa sabirnice pomoću funkcije `read_byte()` neposredno ovisne o funkciji `read_bit()`. Slika 15 prikazuje vremenski dijagram čitanja jednog bita sa sabirnice.

Slika 19 prikazuje dijagram toka funkcije za iniciranje temperaturnog mjerenja i vraćanje rezultata. Funkcija se sastoji od 2 koraka u kojima šalje različite naredbe DS18B20 senzoru. Nakon provjere prisutnosti uređaja mikrokontroler emitira `SKIP_ROM` naredbu. Navedena naredba se može koristiti samo u situaciji kada je jedan uređaj spojen na zajedničku sabirnicu i provjera ROM kodova za adresiranje nije nužna. Primanjem `CONVERT_T` naredbe senzor započinje postupak mjerenja temperature. Vrijeme čekanja mikrokontrolera na analogno-

digitalnu pretvorbu i upis nove vrijednosti u SRAM senzora ovisno je o rezoluciji mjerenja. Funkcija `conversion_wait()` na temelju predane vrijednosti rezolucije vraća potrebno vrijeme čekanja mikrokontrolera. Onemogućavanje nastavka komunikacije realizirano je blokiranjem izvođenja glavnog zadatka (engl. *main task*). Adresiranje uređaja u drugom koraku komunikacije ostalo je isto, ali se šalje nova funkcijska naredba senzoru. Primanjem naredbe `READ_SCRATCHPAD` DS18B20 senzor počinje emitirati sadržaj svojih internih registara na zajedničku sabirnicu počevši od najmanje značajnog bajta (engl. *Least Significant Byte*, LSB). Struktura memorijskog prostora senzora prikazana je slikom 18 (Slika 18).

Zadnji bajt pročitane *Scratchpad* memorije sadrži CRC kod izračunat od strane senzora. S pomoću ovog koda i ekvivalentnog polinoma CRC funkcije možemo prepoznati greške u prijenosu. Postupak detekcije grešaka svodi se na implementaciju CRC generatora prikazanog slikom 16 (Slika 16) i ponovno računanje vrijednosti na *master* strani. Komunikacija je prošla bez greške kada je vrijednost ponovno izračunatog CRC-a jednaka onoj poslanoj od strane uređaja. Provjera kôda implementirana je u funkciji `crc_checker()`.



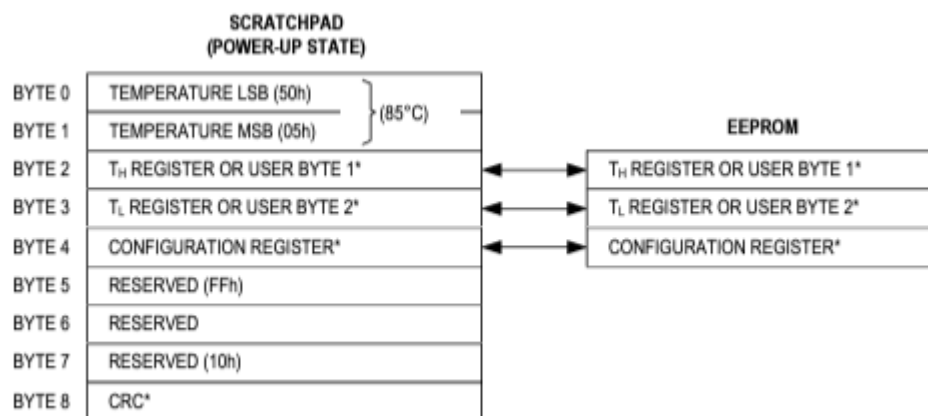
Slika 16. CRC generator. Preuzeto iz [19]

Nakon potvrde valjanosti pridošlih podataka mikrokontroler računa vrijednost mjerenja. Podatci o iznosu i predznaku nalaze se u prva dva registra *Scratchpad* memorije prikazana slikom 17 (Slika 17).

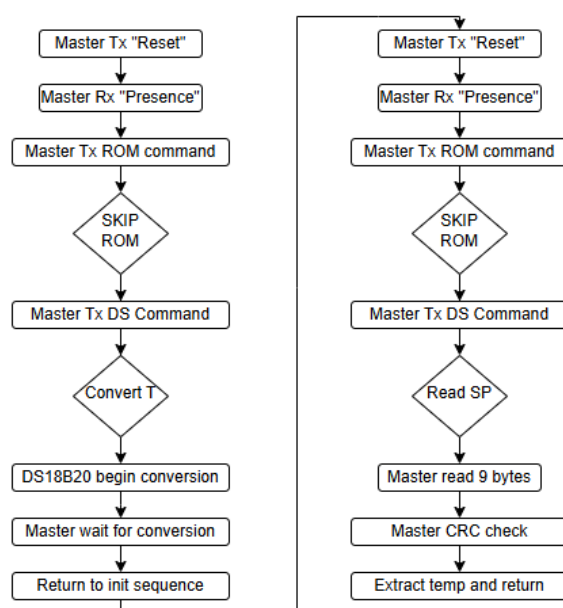
	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴
	BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
MS BYTE	S	S	S	S	S	2 ⁶	2 ⁵	2 ⁴

S = SIGN

Slika 17. Format temperaturnih registara. Preuzeto iz [19].



Slika 18. Struktura memorije DS18B20 senzora. Preuzeto iz [19].



Slika 19. Dijagram toka funkcije za dohvat temperaturnog mjerenja

Rezolucija temperaturnog mjerenja se mijenja upisom nove konstante u 8-bitni konfiguracijski registar senzora.

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	R1	R0	1	1	1	1	1

Slika 20. Struktura konfiguracijskog registra. Preuzeto iz [19].

Tablica 4 prikazuje ovisnost mjerne rezolucije senzora o vrijednostima R0 i R1 bitova.

Povratna vrijednost funkcije `configuration_register()` je konstanta koju je potrebno upisati u konfiguracijski registar za ostvarivanje željene rezolucije. Uređaj se adresira na isti način kao i prilikom dohвата temperaturnog mjerenja. Nakon adresiranja mikrokontroler emitira naredbu `WRITE_SCRATCHPAD` na zajedničku sabirnicu. Nakon

emitiranje naredbe potrebno je poslati tri bajta. Prvi i drugi poslani bajt upisuju se u interne registre T_H i T_L čije vrijednosti se uspoređuju s izmjerenom temperaturom. Napuštanje temperaturnog intervala $[T_L, T_H]$ rezultira postavljanjem interne zastavice senzora. Mikrokontroler može čitati stanje zastavice naredbom `ALARM_SEARCH`. Treći poslani bajt se upisuje u konfiguracijski registar senzora i mijenja rezoluciju mjerenja.

Tablica 4. Ovisnost rezolucije o stanju konfiguracijskog registra

R1	R0	Rezolucija
0	0	9-bit
0	1	10-bit
1	0	11-bit
1	1	12-bit

4.2. SEN0189 senzor kod

4.2.1. A/D pretvornik (ADC)

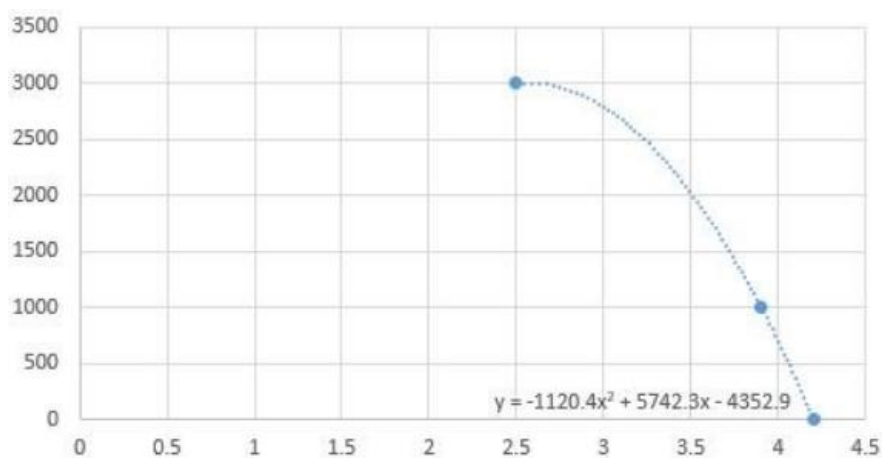
ESP32 sadrži dva analogno-digitalna pretvornika sa sukcesivnom aproksimacijom. Omogućuju očitavanje preko 18 mjernih kanala multipleksiranih na GPIO izvode mikrokontrolera. Postoje tri vrste upravljačkog sklopovlja koje služe optimalnom korištenju pretvornika u različitim načinima rada. Odabir RTC upravljača pruža mogućnost mjerenja u načinu rada niske potrošnje pri nižim brzinama uzorkovanja. Digitalni upravljač optimiran je za visoku protočnost i velike frekvencije signala vođenja. Također, omogućuje izravan upis mjerenih vrijednosti u memoriju pomoću DMA sklopovlja. *Power/Peak detect* upravljač se kontinuirano koristi za uzorkovanje i analogno-digitalnu pretvorbu demoduliranog signala prilikom rada Wi-Fi modula. Vezan je uz SAR ADC 2 i onemogućuje njegovo korištenje tijekom bežične komunikacije.

Podatkovna linija analognog senzora za mjerenje zamućenosti vode SEN0189 spojena je na GPIO izvod D33 mikrokontrolera. Izvod je multipleksiran na ulaz analogno digitalnog pretvornika SAR ADC 1, čime se osigurava mogućnost pretvorbe tijekom rada Wi-Fi modula.

Sklopovlje se inicijalizira pozivom funkcije `turbidityInit()`. Inicijalizacijom mjernog kanala postavljaju se vrijednosti dva važna parametra, atenuacije i rezolucije. Atenuacija povećava mjerno područje ulaznog signala. U projektu je korištena 12-bitna rezolucija i atenuacija od 12dB kojom je osigurana podrška za digitalizaciju ulaznih signala unutar raspona vrijednosti napona od 150 mV – 2450 mV.

Pozivanjem funkcije `turbidityRead()` dohvaća se nova vrijednost razine zamućenosti vode. Vrijednost na izlazu analogno-digitalnog pretvornika dobivena pozivom naredbe `adc_oneshot_read()` pretvara se u ekvivalentni napon ($V_{out} = (D_{out} / D_{max}) * V_{ref}$). Korišteni način pretvorbe nije idealan zbog varijacija u referentnom naponu analogno-digitalnog pretvornika. Informacija o mjeri zamućenosti vode dobiva se preračunavanjem izlaznog napona u NTU jedinice formulom danom od strane proizvođača senzora. Slika 21 prikazuje graf ovisnosti mjerenja zamućenosti o izlaznom naponu.

```
current_reading = -1120.4 * pow(voltage, 2) + 5742.3 * voltage - 4352.9;
```



Slika 21. NTU u ovisnosti o mjerenom naponu. Preuzeto iz [20].

4.2.2. Implementacija filtra za senzor zamućenosti

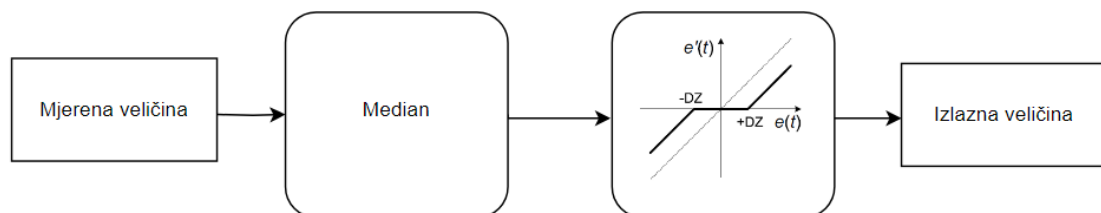
Prolaskom neotopljenih krutina između fotodiode i fotodetektora senzora dolazi do kratkotrajnog smanjenja transmisije. Posljedica naglog smanjenja osvjetljenja fotodetektora je impulsna promjena mjerene vrijednosti razine zamućenosti vode koja utječe na kontrolni algoritam pumpe. Smanjenje utjecaja impulsnih promjena napravljeno je implementacijom *median* filtra. Filtar zahtjeva veći broj uzoraka za izračun konačne vrijednosti mjerenja. Broj uzoraka i vjerojatnost pojave impulsne smetnje na izlazu su obrnuto proporcionalne

veliĉine. Uzorci se uzimaju u razmaknutim vremenskim trenutcima i nakon prikupljanja zadovoljavajućeg broja sortiraju se po iznosu. Kao konaĉna mjerena vrijednost uzima se vrijednost srednjeg člana sortiranog polja ili, ako imamo paran broj uzoraka, srednja vrijednost dva srednja člana.

Deadzone filter ili filter sa zonom neosjetljivosti uklanja utjecaj malih promjena mjerene veliĉine na izlaznu vrijednost. Male varijacije mjerene vrijednosti posljedica su inherentnog šuma senzora (elektroniĉki šum), svjetlosti iz neuraĉunatih izvora i promjenjivosti uzorka. Navedeni šumovi imaju aditivno djelovanje i mogu dovesti do neopravdane promjene stanja rada pumpe.

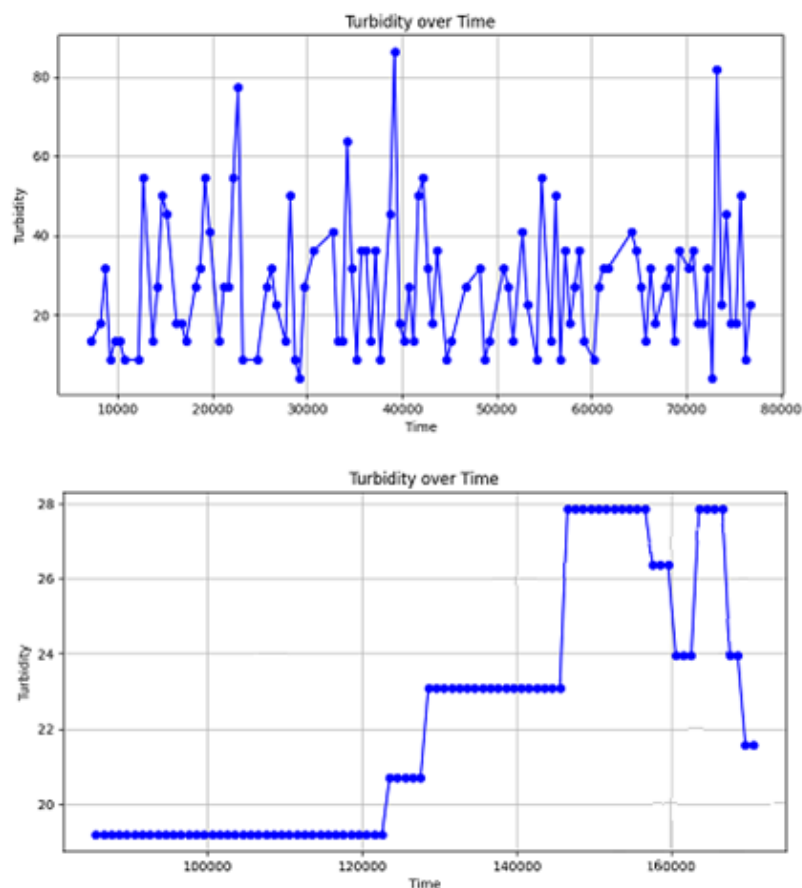
```
if(current_reading > data->reading + DEADZONE){
    data->reading = current_reading - DEADZONE;
} else if(current_reading < data->reading - DEADZONE){
    data->reading = current_reading + DEADZONE;
}
```

Odsječak kôda 1. Implementacija *deadzone* filtra



Slika 22. Filtriranje mjerene veliĉine.

Slika 23 prikazuje mjerene vrijednosti zamućenosti vode prije i poslije implementacije dvostupanjskog digitalnog filtra. Filter stabilizira iznos mjerenja i pokazuje zadovoljavajuće rezultate u uklanjanju impulsnih smetnji.



Slika 23. Mjerene vrijednosti zamućenosti prije/poslije implementacije filtra.

4.3. Upravljanje aktuatorima

Pumpa i višestupanjski filtar tvore sustav za pročišćavanje vode u akvariju. Promjenjivi protok pumpe ovisan je o trenutno izmjerenoj vrijednosti zamućenosti vode. Korištena je tranzistorska sklopka s kapacitivnom zaštitom od visokog induciranog napona prilikom isključenja induktivnog tereta. Pulsno-širinskom modulacijom s promjenjivim radnim ciklusom mijenja se srednja vrijednost napona *gate* elektrode tranzistorske sklopke. Promjenama navedenog napona mijenja se statička radna točka tranzistora koja utječe na R_{DS} otpor u izlaznom krugu. Veći iznosi otpora između *drain* i *source* elektrode smanjuju struju i posljedično protok.

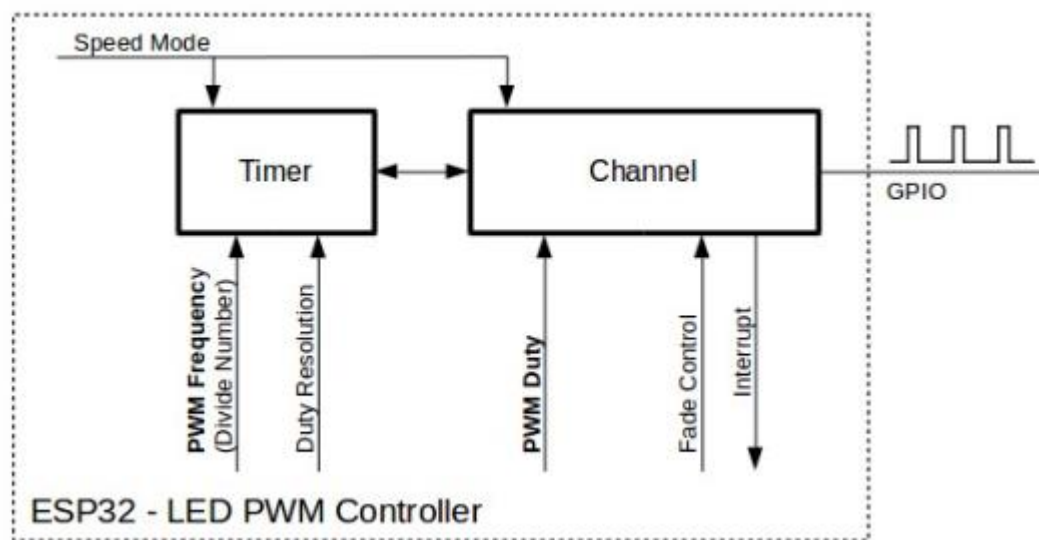


Slika 24. Korištena pumpa i četverostupanjski filter

4.3.1. LEDC

Potreban pulsno-širinski moduliran signal generira se pomoću jedinice za kontrolu intenziteta svjetla LED dioda. Sklop se sastoji od osam brojača i kanala kojima je moguće generirati međusobno neovisne valne oblike. Inicijalizacija sklopa započinje pozivom funkcije `ledc_init()` koja prvo konfigurira korišteni brojač. Parametri poput odabira izvora signala vođenja, indeksa korištenog brojača, rezolucije, frekvencije PWM signala i drugih, upisuju se u pomoćnu strukturu `ledc_timer_config_t`. Struktura se koristi u `ledc_timer_config()` funkciji koja na temelju predanih vrijednosti računa i upisuje nova stanja registara brojača. PWM generator omogućuje postupene prijelaze između različitih radnih ciklusa signala. Navedena funkcionalnost se omogućuje pozivom funkcije `ledc_fade_func_install()` deklarirane u `driver/ledc.h` dokumentu.

Nakon brojača, konfigurira se PWM kanal na kojem se generira signal. Za upis željenih vrijednosti parametara koristi se pomoćna struktura `ledc_channel_config_t` koja se predaje funkciji za izračun i upis novih stanja u registre `ledc_channel_config`. Blokovska shema PWM generatora prikazana je slikom (Slika 25).



Slika 25. Struktura LED PWM upravljača. Preuzeto iz [23].

5. Bežično slanje mjerenih podataka

5.1. Wi-Fi

Wi-Fi je skup protokola za bežičnu komunikaciju temeljenih na IEEE 802.11 standardima. Razmjena podataka vrši se pomoću elektromagnetskih valova u radiofrekvencijskom spektru. Najčešće korišteni frekvencijski pojasevi su ISM (2400-2483.5 MHz) i OD87 (5470-5725 MHz). Od pojave prve IEEE 802.11 norme 1997. godine pa do danas pojavile su se brojne inačice norme koje koriste različite frekvencijske pojaseve, modulacije, zaštitna kodiranja i multipleksiranja za postizanje što većih brzina prijenosa uz male vjerojatnosti pogreške. Tablica 5 prikazuje IEEE 802.11 norme podržane od strane ESP32 mikrokontrolera. Korištene su 1x1 MIMO tehnike pri uporabi 802.11n norme.

Tablica 5. izdvojene IEEE 802.11 norme

Inačica norme	Frekvencijsko područje	Max brzina prijenosa	Godina uvođenja
802.11b	2,4 GHz	11Mbit/s	1999.
802.11g	2,4 GHz	11Mbit/s (HR-DSSS) 54 Mbit/s (OFDM)	2001.
802.11n	2,4/5 GHz	600 Mbit/s	2009.

Različitom konfiguracijom Wi-fi drivera mikrokontroler može obavljati funkcije pristupne točke, pristupne stanice ili oboje istovremeno. Korišten je način rada pristupne stanice u kojem se ESP32 spaja na pristupnu točku.

Postupak uspostave veze sastoji se od nekoliko koraka. Prvi korak je inicijalizacija *lightweight* TCP/IP stoga i stvaranje LwIP zadatka zaduženog za provođenje operacija složaja. Sve navedene operacije obavlja funkcija

```
ESP_ERROR_CHECK(esp_netif_init())
```

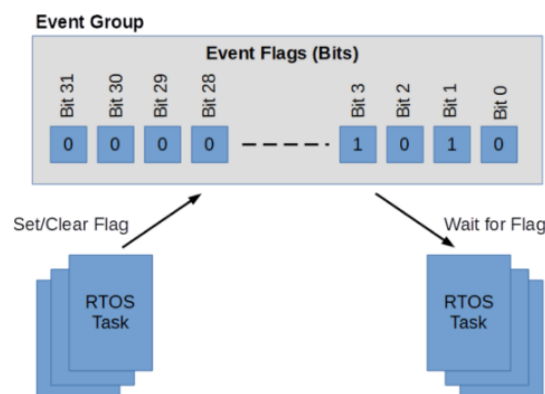
Funkcija je zaokružena *macro* naredbom kojom se izvođeni program terminira u slučaju da je povratna vrijednost funkcije *ESP_FAIL*. Nakon omogućavanja složaja pozivamo funkciju za stvaranje Wi-Fi zadatka i inicijalizaciju Wi-Fi drivera na temelju *default* konfiguracije

pristupne stanice. Dodatno, registriramo *handler* funkcije u glavnoj event petlji za obradu događaja koje generira Wi-Fi driver.

Pozivom funkcije `esp_wifi_start()` Wi-Fi driver objavljuje događaj `WIFI_EVENT_STA_START` čiju obradu provodi *event-handler* namijenjen za Wi-Fi događaje. Dolaskom događaja za početak uspostave veze *handler* poziva funkciju `esp_wifi_connect()` kojom započinje postupak skeniranja, spajanja na AP i inicijalizacije DHCP klijenta za dohvaćanje IP adrese. U slučaju uspješne dodjele IP adrese od strane DHCP servera objavljuje se događaj `IP_EVENT_STA_GOT_IP` na *default event loop*. Pojava ovog događaja predstavlja završetak postupka uspostave veze s AP-om. *Event-handler* koji obrađuje ovaj događaj dojavljuje ostalim zadacima da je postupak spajanja gotov.

Uspješnost postupka spajanja dojavljuje se pomoću FreeRTOS grupe događaja (engl. *Event Group*) prikazane slikom (Slika 26). Iskorištena su ukupno 2 bita unutar korištene grupe koji služe za prijenos informacije o uspješnosti spajanja. U slučaju neuspješnog povezivanja generira se događaj `WIFI_EVENT_STA_DISCONNECTED` nakon čega se ponovno pokušava uspostaviti veza maksimalno `_MAXRETRY` puta prije nego se postavi `WIFI_FAIL` bit grupe događaja.

```
#define WIFI_SUCCESS BIT0
#define WIFI_FAIL BIT1
static EventGroupHandle_t wifi_event_group;
```

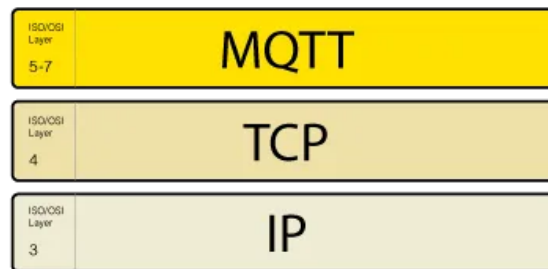


Slika 26. FreeRTOS *event group* i upravljanje izvođenjem. Preuzeto iz [8].

Postavljeni bitovi koriste se u funkciji `wait_wifi()` koju poziva glavni zadatak i kojom se blokira njegovo daljnje izvođenje. Blokiranje izvođenja omogućeno je pozivom funkcije `xEventGroupWaitBits()` koja čeka na promjenu jednog od bitova, `WIFI_FAIL` ili `WIFI_SUCCESS`, u grupi događaja.

5.2. MQTT

MQTT (*Message Queuing Telemetry Transport*) je *lightweight* protokol koji implementira *pub/sub* model komunikacije. Optimiran je za slanje paketa nesigurnim mrežama s niskom kvalitetom veze i ograničenim protokom. MQTT je protokol viših slojeva ISO/OSI modela (5-7) i zahtjeva postojanje TCP/IP složaja na svim strankama koje žele sudjelovati u komunikaciji.



Slika 27. MQTT u ISO/OSI modelu. Preuzeto iz [9].

Dvije komponente nužne za povezivanje i razmjenu podataka su MQTT *client* i *broker*. MQTT *broker* je centralna jedinica za primanje poruka od klijenta i njihovo usmjeravanje zainteresiranim stranama. Ne postoji direktna komunikacija između dva različita klijenta. MQTT klijent je svaki uređaj koji implementira biblioteku protokola, ima TCP/IP složaj i povezan je s *brokerom*. Model razmjene poruka temelji se na temama i pretplatama. Teme su nizovi znakova pomoću kojih se određuju zainteresirani klijenti i usmjerava poruka od strane *brokera*. Nakon uspostave veze uređaji mogu objavljivati poruke i pretplaćivati se na događaje njima od interesa. Filtriranje poruka ovisno o temi predanoj u strukturi poslanog paketa radi *broker*.

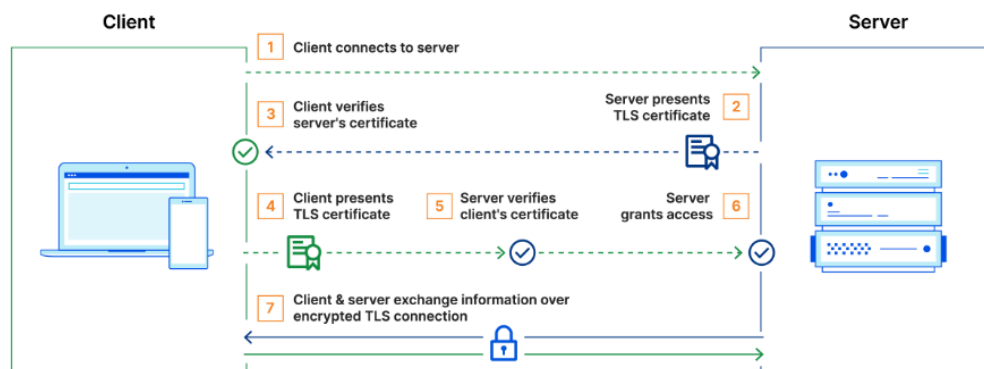


Slika 28. Struktura *Publish* i *Subscribe* paketa. Preuzeto iz [10].

5.2.1. ESP-MQTT

ESP-MQTT je implementacija MQTT protokol klijenta na ESP32 mikrokontroleru. Postupak uspostave veze između servera s implementiranom MQTT broker funkcionalnošću i uređaja započinje pozivom funkcije `mqtt_app_start()`. Navedena funkcija unosi parametre u pomoćnu strukturu `esp_mqtt_client_config_t`. Funkcija `esp_mqtt_client_init()` putem pokazivača na konfiguracijsku strukturu pristupa potrebnim informacijama za inicijalizaciju MQTT klijenta. Pomoću funkcije `esp_mqtt_client_register_event()` registriramo *event-handler* za MQTT događaje koji se objavljuju na *event-loop* stvoren tijekom konfiguriranja. Naredba `esp_mqtt_client_start()` započinje postupak spajanja klijenta. Funkcija stvara novi FreeRTOS zadatak u kojem je implementirana potrebna funkcionalnost spajanja i održavanja veze između mikrokontrolera i *brokera*.

Sva razmjena podataka između mikrokontrolera i oblaka u projektu se odvija preko MQTT protokola osiguranog pomoću TLS-a (engl. *Transport Layer Security*). mbedTLS je implementacija TLS-a namijenjena za ugradbene sustave s ograničenim resursima. TLS stog zauzima ispod 50kB u RAM-u i podržava brojne kriptografske algoritme (npr. RSA za asimetrične ključeve, SHA-1 i SHA-2 za hash funkcije, ...). Implementirana je međusobna autentikacija od strane servera i uređaja. Certifikati za međusobnu autentikaciju predani su klijentu kao elementi inicijalizacijske strukture spomenute u prijašnjem odlomku.



Slika 29. TLS *handshake* i uspostava komunikacije između uređaja i servera. Preuzeto iz [24].

Ključevi i certifikati nalaze se u poddirektorijima *certificates* i *keys* projekta. Ugrađuju se u .bin datoteku i nalaze se u .rodata dijelu *flash* memorije. *Linker* generira simbolička imena pomoću kojih je omogućen pristup memorijskim lokacijama na kojima se nalaze podatci.

Ugrađivanje korištenih ključeva i certifikata u *flash* memoriju realizirano je odsječkom kôda (Odsječak kôda 2) koji se dodaje u glavnu CMake konfiguracijsku datoteku projekta.

```
target_add_binary_data(main.elf "main/certificates/brokerCert.pem" TEXT)
target_add_binary_data(main.elf "main/keys/deviceCert.key" TEXT)
target_add_binary_data(main.elf "main/certificates/deviceCert.pem" TEXT)
```

Odsječak kôda 2. Ugrađivanje korištenih ključeva i certifikata u *flash* memoriju

○ Total sizes:	○ Total sizes:
Used static DRAM: 33828 bytes (146908 remain, 18.7% used)	Used static DRAM: 33828 bytes (146908 remain, 18.7% used)
.data size: 15524 bytes	.data size: 15524 bytes
.bss size: 18304 bytes	.bss size: 18304 bytes
Used static IRAM: 93126 bytes (37946 remain, 71.0% used)	Used static IRAM: 93126 bytes (37946 remain, 71.0% used)
.text size: 92099 bytes	.text size: 92099 bytes
.vectors size: 1027 bytes	.vectors size: 1027 bytes
Used Flash size : 812159 bytes	Used Flash size : 818087 bytes
.text: 646019 bytes	.text: 647023 bytes
.rodata: 164984 bytes	.rodata: 171608 bytes
Total image size: 920009 bytes (.bin may be padded larger)	Total image size: 927537 bytes (.bin may be padded larger)

Slika 30. Promjena veličine `.rodata` dijela *flash* memorije nakon ugradnje certifikata

```
extern const uint8_t brokerCert_pem_start[]
asm("_binary_brokerCert_pem_start");
extern const uint8_t brokerCert_pem_end[]
asm("_binary_brokerCert_pem_end");
```

X.509 certifikati generirani su koristeći naredbe definirane u *OpenSSL* biblioteci. Nakon međusobne autentikacije i spajanja na poslužitelj uređaj može početi objavljivati poruke na određene teme. Formiranje poruke i njeno slanje implementirano je u funkciji `send_telemetry_data()`. Argumenti naredbe `esp_mqtt_client_publish()` su tema na koju klijent želi objaviti poruku, podatci za objavu, duljina podatka (važno u slučaju velikih paketa i moguće potrebe za segmentacijom), QoS (engl. *Quality of Service*), *retain* zastavica.

```
static void send_telemetry_data(TURBIDITY_reading *turbidity,
DS18B20_reading *temperature){
    uint16_t length_temp = snprintf(NULL, 0, "%f",
temperature->reading) + snprintf(NULL, 0, "%d", turbidity-
>reading) + 1 + 1;
    ESP_LOGI("LIST: ", "list size is %d", length_temp);
    char data_string[length_temp];
    sprintf(data_string, "%f|%d", temperature->reading,
turbidity->reading);

    esp_mqtt_client_publish(mqtt_client, "telemetry/my-
device/temperature", data_string, 0, 0, 0);
}
```

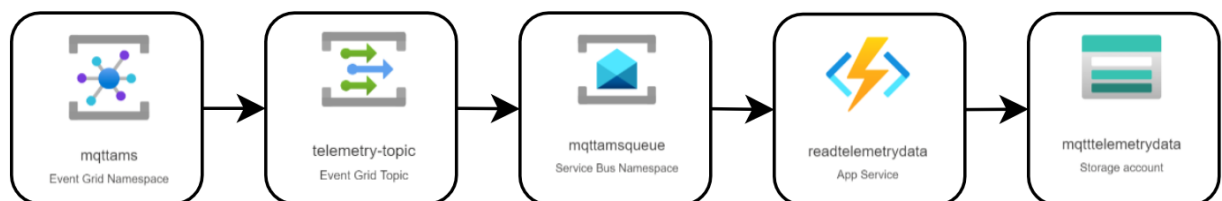
Odsječak kôda 3. Funkcija za slanje mjerene temperature i zamućenosti

5.3. Azure

Microsoft Azure je platforma za računarstvo u oblaku koja omogućava razvoj, upravljanje i implementaciju aplikacija kroz globalnu mrežu podatkovnih centara. Pruža širok spektar usluga poput virtualnih strojeva, baza podataka i brojnih drugih.

U projektu su koristi pet različitih usluga Azure platforme: *Azure Event Grid*, *Service Bus*, *Function App*, *Table Storage*, *Event Grid Topic*.

Slika 31 prikazuje blok shemu i tok podataka implementiranog sustava.



Slika 31. Blok shema mqtamsorg grupe resursa

Azure Event Grid je potpuno upravljana usluga distribucije poruka. Implementira Pub/Sub model komunikacije i podržava protokole HTTP i MQTT. Nakon dvostruke autentikacije koristeći X.509 certifikate uspostavlja se enkriptirani tok podataka između uređaja i servera. Poruke se šalju svakih 10 sekundi i sadrže informacije o trenutnom stanju temperature i

zamućenosti vode u akvariju. Na temelju specificirane teme *broker* usmjerava paket na uređaje ili usluge kojima je on od interesa.

Svakim dolaskom nove poruke u *Service Bus Queue* izvodi se funkcija `readtelemetrydata()`. Rastav ulaznog niza, uklanjanje nepotrebnih informacija, dekodiranje tijela poruke i naknadno spremanje dekodiranih vrijednosti temperature i zamućenosti u *Table Storage* su zadaće navedene funkcije. Mjerene vrijednosti skladištene u tablici podataka prikazane su slikom 33 (Slika 33). Spajanje na server, slanje paketa, njihov dolazak u *Queue* i dekodiranje tijela poruke pomoću *online Base64* dekodera s namjerom provjere prijenosa prikazani su slikom 34 (Slika 34).



Slika 32. *Service Bus Queue*. Preuzeto iz [11].

☐	PartitionKey	Temperature	Turbidity
☐	data	27.0625	234
☐	data	27.0625	227
☐	data	27	231

Slika 33. Mjerene vrijednosti u *mqtttelemetrydata* tablici

```

I (18377) MQTT: Connected to broker
I (14657) LIST: : list size is 14
I (14657) MQTT: Published data Temperature: 27.500000, Turbidity: 100
I (24657) LIST: : list size is 14
I (24657) MQTT: Published data Temperature: 27.500000, Turbidity: 100
I (34657) LIST: : list size is 14
I (34657) MQTT: Published data Temperature: 27.500000, Turbidity: 100
I (44657) LIST: : list size is 14
I (44657) MQTT: Published data Temperature: 27.500000, Turbidity: 100
          
```

Message Body Message Properties

```

{"id":"ac917681-e677-49d3-91b3-c3a6d3a9999","source":"mqtt","type":"MQTT.EventPublished","data_base64":"MQjUjT5wK4Dw4D5wK4M4++","time":"2024-07-06T17:22:33.910Z","specversion":"1.0","datacontenttype":"application/octet-stream","subject":"telemetry/ESP32-node/temperature"}
          
```

Slika 34. Testiranje komunikacije između mikrokontrolera i *brokera*.

6. Pokazni model

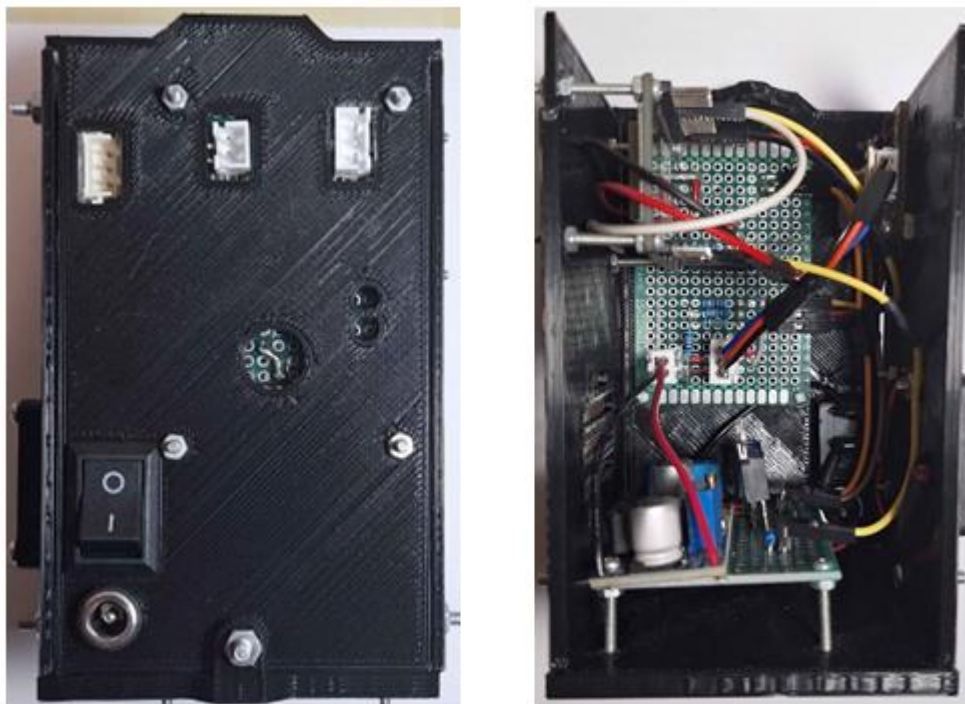
6.1. Opis sklopovlja

Pokazni model sastoji se od malog akvarija namijenjenog za prijenos riba, upravljačke jedinice i kućišta za filter i senzore. Pokazni model prikazan je slikom (Slika 35).



Slika 35. Pokazni model

Sustav koristi dva različita napona napajanja. Mrežnim adapterom se izmjenični napon gradske mreže pretvara u 12V DC. Mikrokontroler i senzori su napajani preko propusnog regulatora s prekidanjem struje koji smanjuje iznos istosmjernog napona s 12V na 5V. Vodena pumpa za filtracijski sustav se spaja izravno na 12V. GPIO izvod D18 mikrokontrolera spojen je preko 470 Ω otpornika na upravljačku elektrodu tranzistorske sklopke za pogon vodene pumpe. Pomoću dvo- i tropinskih JST konektora olakšano je spajanje senzora na upravljačku jedinicu. Senzorski čvor, vodena pumpa i četverostupanjski filter nalaze se unutar kućišta smještenog u akvariju. Upravljački sklop prikazan je slikom (Slika 36).



Slika 36. Upravljački sklop

6.2. Rezultati

Provedena su ispitivanja rada sustava u trajanju od 12 sati. Mjerene vrijednosti senzora očitavaju se i šalju na oblak (engl. *cloud*) svakih 60 sekundi. Tijekom testiranja sustav je bio izložen raznim situacijama, poput naglog gubitka veze s pristupnom točkom ili serverom, koje mogu ugroziti njegov kontinuiran rad. Sustav je uspješno obradio krizne događaje i nastavio mjerenje i regulaciju interesnih parametara. Nakon ponovne uspostave veze s pristupnom točkom ili serverom uređaj je počeo slati podatke. Utjecaji postupka sedimentacije suspendiranih čestica ne mogu se potpuno ukloniti iz mjerenih vrijednosti zamućenosti vode.

7. Zaključak

Porast broja vlasnika kućnih ljubimaca, posebno riba, naglasio je potrebu za inovativnim rješenjima u održavanju optimalnih uvjeta za vodene organizme. Razvoj sustava za praćenje i kontrolu parametara vode u akvarijima predstavlja značajan korak naprijed u tehnologiji brige o kućnim ljubimcima.

Kroz ovaj rad razvijen je sustav za praćenje i kontrolu temperature i zamućenosti vode u akvarijima, koristeći odgovarajuće senzorske i aktuatorске tehnologije, ugradbeno računalo, i bežičnu vezu za udaljeno praćenje. Omogućen je uvid u trenutno stanje vode i automatska prilagodba kako bi se osigurala stabilnost ekosustava.

Implementirani uređaj sastoji se od ESP32 mikrokontrolera, DS18B20 temperaturnog senzora, SEN0189 senzora za mjerenje razine zamućenosti vode, pumpe i filtra. Popratno sklopovlje osigurava napajanje i kontinuiran rad sustava. ESP32 mikrokontroler omogućuje rad sustava s niskom potrošnjom, Wi-Fi povezanost i nisku cijenu. Programska potpora za ESP32 razvijena je pomoću ESP-IDF razvojnog okvira. Mjereni podatci se šalju na *Azure Event Grid* koristeći MQTT protokol za komunikaciju između uređaja i oblaka.

Važnost ovog rada je ukazati na probleme pri realizaciji sustava za kontinuirano praćenje i održavanje umjetnih vodenih staništa. Nepristupačane cijene senzora i aktuatora za mjerenje i upravljanje parametrima od interesa godinama su prepeka implementaciji vlastitih rješenja.

Literatura

- [1] <https://www.viclabs.co.uk/formazin-turbidity-standard-40ftu>
- [2] <https://my.hach.com/formazin-turbidity-standard-4000-ntu-100-ml/product?id=59428452407>
- [3] <https://www.az-delivery.de/products/esp-32-dev-kit-c-v4?shpxid=d4cec241-3c15-4d36-aa99-05a4133a0e42>
- [4] https://mirrobo.ru/wp-content/uploads/2016/11/Cadence_Tensillica_Xtensa_LX6_ds.pdf
- [5] <https://intaqo.com/>
- [6] <https://aquariumopstarten.nl/blog/intaqo-controller-review/>
- [7] <https://www.tinytronics.nl/en/communication-and-signals/wireless/wi-fi/modules/esp32-wroom-32e-wi-fi-and-bluetooth-module>
- [8] <https://open4tech.com/communication-between-rtos-tasks/>
- [9] <https://www.hivemq.com/blog/how-to-get-started-with-mqtt/>
- [10] <https://www.hivemq.com/blog/mqtt-essentials-part-3-client-broker-connection-establishment/>
- [11] <https://microsoft.github.io/AzureTipsAndTricks/blog/tip327.html>
- [12] <https://www.freertos.org/FreeRTOS-quick-start-guide.html>
- [13] <https://aquariumopstarten.nl/blog/intaqo-controller-review/>
- [14] Alon Fliess, *Azure Event Grid MQTT Broker in Practice*, (2024.), Poveznica: <https://medium.com/@alonfliess/article-two-of-five-azure-event-grid-mqtt-broker-in-practice-89918f8287d2>
- [15] Helene Volkoff and Ivar Rønnestadt, *Effects of temperature on feeding and digestive processes in fish*, (2020.), Poveznica: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7678922/>
- [16] Jo-Anne E. Cavanaugh, Kristy Lee Hogsden, Jon S. Harding: *Effects of suspended sediments on freshwater fish*, (2014.), Poveznica: <https://envirolink.govt.nz/assets/Envirolink/1445-WCRC129-Effects-of-suspended-sediment-on-freshwater-fish.pdf>
- [17] *Mjerenje mutnoće – turbidimetrija i nefelometrija*, Nenad Bolf, Fakultet kemijskog inženjerstva i tehnologije Sveučilišta u Zagrebu, (2020.), poveznica: <https://hrcak.srce.hr/file/360391>
- [18] Melvin Bender, Erkin Kirdan, Marc-Oliver Pahl, George Carle: *Open-Source MQTT Evaluation, Conference: 2021 IEEE 18th Annual Consumer Communications & Networking Conference*, Poveznica: https://www.researchgate.net/publication/350000267_Open-Source_MQTT_Evaluation

- [19] Analog Devices, *DS18B20 temperature sensor datasheet*, (2020.), Poveznica:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>
- [20] Dfrobot, *SEN0189 water turbidity sensor*, Poveznica:
https://wiki.dfrobot.com/Turbidity_sensor_SKU_SEN0189
- [21] ESP32 Datasheet
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [22] ESP32 Technical reference manual
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [23] <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/ledc.html>
- [24] <https://www.cloudflare.com/en-gb/learning/access-management/what-is-mutual-tls/>
- [25] Michelle Megna, *Forbes, Pet Ownership Statistics 2024*
<https://www.forbes.com/advisor/pet-insurance/pet-ownership-statistics/>

Sažetak

Sustav za praćenje i održavanje kakvoće vode u akvariju

U okviru rada razvijen je sustav za praćenje i održavanje okolišnih uvjeta u akvariju. Prate se parametri temperature i zamućenosti vode, a stanje se upravlja pomoću pumpe i filtra. Implementirani sustav sastoji se od ESP32 mikrokontrolera, temperaturnog senzora DS18B20, senzora za mjerenje razine zamućenosti vode SEN0189, pumpe i filtra. ESP32 mikrokontroler omogućuje rad sustava s niskom potrošnjom, Wi-Fi povezivost i nisku cijenu. Mikrokontroler prikuplja, obrađuje i bežično odašilje podatke mjerene sensorima te upravlja aktuatorima. Programska potpora za mikrokontroler razvijena je korištenjem ESP-IDF razvojnog okvira, a podaci se šalju na Azure Event Grid koristeći MQTT protokol. Sigurnost komunikacije između mikrokontrolera i servera povećana je korištenjem TLS protokola (MQTT over TLS, MQTTS).

Ključne riječi: akvarij, DS18B20, senzor zamućenosti vode, 1-Wire, ADC, Azure, MQTT, kontinuirano mjerenje, regulacija

Summary

System for monitoring and water quality maintenance in the aquarium

In this thesis a system for monitoring and control of the environmental conditions in the aquarium was developed. The temperature and turbidity of the water are monitored parameters, while the conditions are controlled using a pump and filter. The system consists of an ESP32 microcontroller, a DS18B20 temperature sensor, a SEN0189 water turbidity sensor, a pump and a filter. The ESP32 microcontroller enables low power operation, Wi-Fi connectivity and low cost. The microcontroller acquires, processes and wirelessly transmits the data measured by the sensors and controls the actuators. The microcontroller firmware was developed using the ESP IDF development framework, while sensor data is sent to Azure Event Grid using the MQTT protocol. The security of communication between the microcontroller and the server is increased by using the TLS protocol (MQTT over TLS, MQTTS).

Keywords: aquarium, DS18B20, water turbidity sensor, 1-Wire, ADC, Azure, MQTT, continuous measurement, regulation